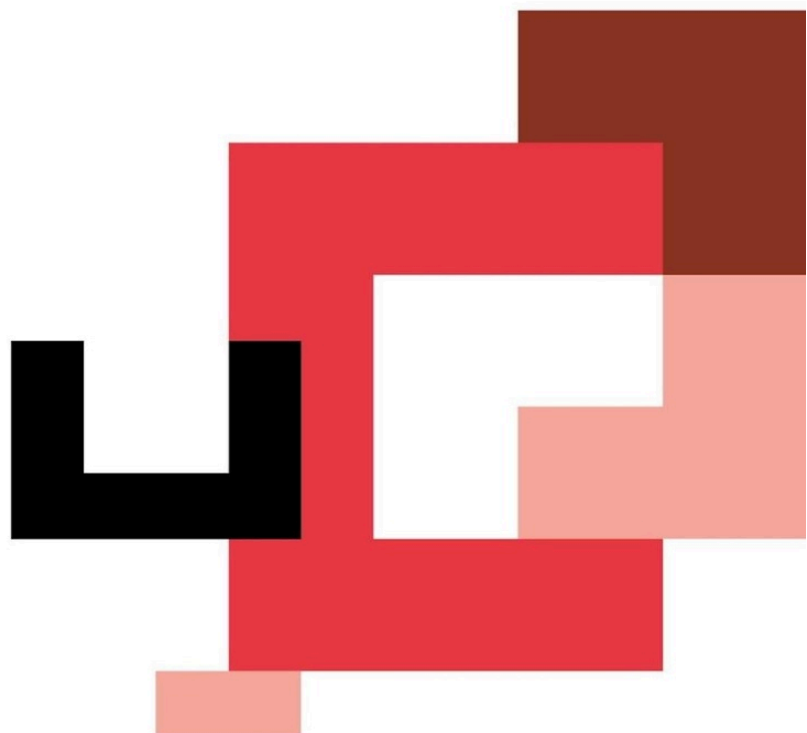




FULL STACK DEVELOPER + IA

Módulo 3 | Feature 2

Conexión con API real +
Formularios + Custom Hooks



Objetivo de la feature

Conectar el frontend con el backend real, entendiendo:

- Cómo consumir una API con **Axios**
- Cómo gestionar datos asíncronos con **useEffect**
- Cómo encapsular lógica en **custom hooks**
- Cómo manejar estados de **loading y error**
- Cómo crear **formularios controlados**
- Cómo interactuar con el DOM usando **useRef**

Importante

Esta feature empezó en el Sprint 1 y continúa aquí.

En este sprint:

- Sustituimos los datos mock por datos reales
- Seguimos construyendo parte de la UI
- Añadimos nuevas funcionalidades (auth, reviews, hooks)

Aquí tienes el ejemplo que puedes tomar como guía. Navega por la página:

<https://shop-example-react.netlify.app/sprint-2/>

Requisitos

Tech / configuración

- Backend corriendo en:

Shell

```
http://localhost:3000
```

- React + Vite
- Axios instalado

Shell

```
npm install axios
```

Estructura sugerida

Shell

src/

api/

 axios.js

 products.js

 reviews.js

 auth.js

hooks/

 useProducts.js

 useProduct.js

 useReviews.js

styles/

 variables.css

 index.css

components/

 Layout/

 Header/

 Footer/

 ProductCard/

 ProductGrid/

 StarRating/

 ReviewList/

```
  FormInput/  
  Button/  
pages/  
  HomePage/  
  ProductsPage/  
  ProductDetailPage/  
  LoginPage/  
  RegisterPage/  
  NotFoundPage/  
router/  
  index.jsx  
App.jsx  
main.jsx
```

Rutas obligatorias

Método	Ruta	Descripción
GET	/	Home
GET	/products	Catálogo
GET	/products/:id	Detalle + reviews

GET	/login	Login
GET	/register	Registro
GET	*	404

Qué hay que hacer (paso a paso)

1) Crear instancia global de Axios

En `src/api/axios.js`:

- Crear instancia con:

```
JavaScript
baseUrl: 'http://localhost:3000'
```

Objetivo: centralizar todas las llamadas HTTP.

2) Crear capa de API

Separar llamadas por dominio:

products.js

- `getProducts()`
- `getProductById(id)`

reviews.js

- `getReviews(productId)`

auth.js

- `login(data)`
- `register(data)`

Objetivo: no llamar a Axios directamente desde componentes.

3) Crear custom hooks

useProducts

- Fetch al montar
- useState + useEffect

useProduct

- Fetch por id

useReviews

- Fetch por producto

Todos deben devolver:

```
JavaScript  
{ data, loading, error }
```

Objetivo: reutilizar lógica y limpiar componentes.

4) Sustituir mock por API real

- Eliminar `mockProducts.js`
- Usar hooks en páginas:

HomePage

- useProducts()

ProductsPage

- useProducts() + filtro

ProductDetailPage

- useProduct(id)
- useReviews(id)

Objetivo: datos reales.

5) Gestionar estados

Cada pantalla debe manejar:

- loading → mostrar estado de carga
- error → mostrar mensaje

Objetivo: UX realista.

6) Crear componentes nuevos

StarRating

- Mostrar estrellas según rating

ReviewList

- Mostrar lista de reviews

Este es opcional por el momento. Mientras no tengamos usuarios logueados no pondremos esto en activo.

Button

- Variantes: primary, secondary, danger

Recuerda que puedes hacer un botón base y cambiarle las props dependiendo la necesidad.

FormInput

- label + input + error
- reutilizable

Objetivo: UI más completa y reutilizable.

7) Crear formularios

LoginPage

- Formulario controlado
- value + onChange
- Validación básica
- Llamada a login()

RegisterPage

- Formulario controlado
- Validación (password, confirmación)

Objetivo: primera interacción real con backend.

8) Usar useRef

En FormInput:

- autofocus en primer input

Objetivo: acceso directo al DOM sin re-render.

Uso de la IA en este sprint

Para guiarte

- Explicación de async/await
 - Revisión de hooks
 - Validación de formularios
 - Debug de peticiones
-

Prompts útiles

Custom hooks

Shell

Estoy creando un custom hook en React para hacer fetch con useEffect.

Quiero devolver { data, loading, error }.

¿Estoy siguiendo una buena estructura?

Axios

Shell

Estoy usando axios.create con baseUrl.

¿Cómo organizo mis llamadas para no repetir código?

Formularios

Shell

Estoy haciendo un formulario controlado en React.

¿Cómo valido inputs antes de enviar al backend?

Pistas

useEffect

- Se ejecuta al montar (`[]`)
- Dentro va la función `async`

Estados

Siempre:

JavaScript

`loading` → `true` al inicio

`error` → `null` al inicio

Flujo típico

JavaScript

`try` → petición

`catch` → error

`finally` → `loading false`

Formularios

- Cada input tiene:
 - value
 - onChange
-

Importante que tienen que tener en cuenta

Separación de capas

- api → llamadas HTTP
 - hooks → lógica
 - pages → UI
-

No mezclar lógica

No hacer fetch directamente en componentes.

Backend obligatorio

Si no funciona → revisar backend.

Manejar errores

No asumir que todo funciona.

UI sigue evolucionando

Este sprint también incluye mejoras visuales.

Checks rápidos (autoevaluación)

- Backend responde correctamente
- /products muestra datos reales
- /products/:id muestra detalle

- Reviews se cargan
 - Loading funciona
 - Error se muestra si falla
 - Login envía datos
 - Register valida correctamente
-

Qué pasará en el Sprint 3

- Gestión de estado global (Redux Toolkit)
 - Persistencia de sesión
 - Rutas privadas
 - Carrito y wishlist
-

Enfoque del sprint

- Entender el flujo real frontend-backend
- Separar responsabilidades correctamente
- Construir base escalable

Este sprint es clave: aquí pasas de maqueta a aplicación real.

